

Лабораторная работа №3. Сцены, Анимации, Действия и Камеры.

Цель работы: Отрефакторить и улучшить текущий код движка, добавить в движок функционал смены сцен, обработки действий по пользовательскому вводу, реализовать в движке текстурные анимации и разные виды камер.

Срок: 16.05.2026 (на 25 баллов). Далее баллы урезаются вдвое.

Допустимое количество студентов в группе: Рекомендуемое – 2. Максимальное – 3 человека (со всеми допами как обязательными – помечены * в описании).

Допустимые инструменты: Как во второй лабораторной работе.

Описание: Необходимо реализовать игру с геймплеем в стиле Super Mario Bros с использованием своего движка с ECS фреймворком и необходимым функционалом:

- **Общее:**
 - Необходимо отрефакторить код движка из второй ЛР в соответствии с 7 лекцией и выделить классы GameEngine и Scene.
 - Весь пользовательский ввод должен обрабатываться через Action'ы и InputManager.
 - API движка на примере описан [здесь](#).
 - Необходимо реализовать два типа камеры: Default и FollowX. Для этого под каждый тип камеры заводится свой компонент-тег и своя система, которая обрабатывает сущность с компонентом камеры, когда на ней есть соответствующий тег.
 - Для удобства настройки уровня в дальнейшем используем размеры окна кратные 64 пикселям (см. про сетку [ниже](#)). Например: 1280x768.
- **Сцены:**
 - В игре должно быть выделено две сцены: сцена меню и сцена игры. Для каждой из этих сцен должен быть реализован свой класс наследник от базового класса Scene. Сцены должны переключаться из класса GameEngine.

- Сцена меню:
 - В сцене меню используется Default камера.
 - В сцене меню должны быть две большие кнопки «**Play**» и «**Exit**». Допускается, чтобы кнопки были просто текстом на черном фоне.
 - Кнопка «Play» загружает сцену с игрой.
 - Кнопка «Exit» закрывает игру.
 - *Украсить сцену меню задним фоном, названием игры. Кнопки должны быть оформлены в формате кнопок с фоном и подсветкой при наведении на них мышкой.
- Сцена игры:
 - При загрузке сцены игры считывается файл конфигурации с уровнем и всем объектам (сущностям) из этой конфигурации назначаются соответствующие компоненты.
 - В файле конфигурации уровня позиции объектов задаются в координатах сетки, где 1 ячейка = 64x64 пикселя (см. про сетку [ниже](#)). Поэтому необходимо перевести координаты сетки в координаты пикселей при загрузке объектов.
 - В сцене игры используется FollowX камера.
 - Фон сцены игры должен быть заполнен цветом: #6464ff.
- Активы:
 - Объекты в игре отображаются с помощью различных текстур и анимаций, которые называем "ресурсами" или активами (включая шрифты).
 - Ресурсы загружаются один раз при старте программы и хранятся в классе AssetManager, который, в свою очередь, хранится в классе GameEngine.
 - При создании сущностей и навешивании на них соответствующих компонентов, ссылки на данные в компонентах (SpriteComponent, AnimatorComponent) заполняются из класса AssetManager.
 - Все имена активов в коде должны быть вынесены в отдельные глобальные статические **константы**, для того, чтобы не использовать сырые строки везде в остальных местах в коде.

- Игрок:
 - Игрок в игре представлен Мегаменом, у которого есть несколько разных анимаций: **Idle**, **Бег**, **Прыжок** и **Выстрел**. Необходимо определить в каком состоянии игрок находится и применить соответствующую анимацию.
 - Управление персонажем следующее:
 - Влево – **A**
 - Вправо – **D**
 - Прыжок – **W**
 - Выстрел – **Пробел**
 - Игрок может двигаться влево-вправо и стрелять в любое момент игры. То есть он может двигаться влево-вправо в воздухе (при этом не должна запускаться анимация движения).
 - Прыгать можно только если игрок стоит на платформе.
 - Удерживание клавиши прыжка не должно вызывать повторные прыжки – один раз нажали – один раз прыгнули.
 - Спрайт игрока должен поворачиваться в сторону, куда он движется.
 - Пули летят в ту сторону, куда смотрел игрок при выстреле.
 - Игрок сталкивается с объектами типа `Tile` и не может проходить через них.
 - Игрок НЕ сталкивается с объектами типа `Decor` (декорации).
 - Если игрок падает ниже экрана – он должен возродиться в начале уровня.
 - Если игрок дошел до финиша – его возвращает в сцену меню.
 - У игрока должен быть компонент гравитации (`GravityComponent`), который ускоряет его падение, пока он не столкнётся с плиткой.
 - У игрока есть максимальная скорость, заданная в файле конфигурации – её нельзя превышать по *x* или *y*.
 - Значения `BoundingBoxComponent`'а игрока задается в файле конфигурации.
 - Спрайт и бокс игрока должны быть центрированы относительно его позиции.
 - *Добавляется время прохождения уровня. Его отсчет стартует в момент загрузки сцены с уровнем и заканчивается при пересечении финиша. Время выводится в HUD в левом верхнем углу окна. Лучшее время прохождения уровня должно

выводиться отдельно на сцене меню и должно сохраняться между перезапусками приложения.

- Анимации:

- Для реализации анимаций выделяется отдельная система `AnimationSystem`, которая обновляет кадры анимации.
- Анимации могут быть как зацикленными, так и однократными.
- Анимации при загрузке из конфигурационного файла ассетов упаковываются в класс `Animation`, который хранит `sf::Sprite` всей анимации и необходимые конфигурационные значения (имя, количество кадров анимации, длительность одного кадра анимации в игровых кадрах).
- Для хранения текущего состояния анимации также выделяется отдельный компонент `AnimationStateComponent`, куда в том числе записывается новый стейт анимации, и проверка на то надо ли сменять анимацию или нет.
- Сама смена состояния анимации производится также в отдельной системе (или системах), в которой проверяются необходимые условия для анимации.
- Для хранения стейтов анимации сущности также выделяется отдельный компонент `AnimatorComponent`, в котором в словаре (`map<string, Animation*>`) лежат все анимации конкретной сущности. При смене стейта анимации – сами анимации берутся из этого компонента.

- «Декорации»:

- Объекты типа `Decor` отображаются на экране, но не взаимодействуют ни с чем.

- Тайлы:

- `Tile` – это объекты, определяющие геометрию уровня и взаимодействующие с игроком.
- Тайлы могут быть также анимированы.
- Значения `BoundingBoxComponent`'а каждого тайла равны размеру своего спрайта.
- Тайлы могут быть трех типов: **обычные**, **«кирпичные»** и **«вопросики»** (как доп).

- Кирпичики:
 - Когда кирпичный тайл коллайдится с пулей или игроком **снизу**:
 - На месте кирпичного тайла должна проиграться одноразовая анимация взрыва.
 - Сам тайл и его сущность целиком уничтожается.
 - По истечению анимации взрыва ее сущность тоже должна уничтожиться.
- *Вопросики:
 - Когда тайл с вопросом коллайдится с игроком **снизу**:
 - Создается временная анимированная сущность с монеткой на 30 кадров, на 64 пикселя выше над тайлом с вопросиком.
 - Тайл с вопросом должен замениться на обычный блок.
 - Добавляется счет очков игрока. За каждую выбитую монетку он увеличивается на один. Счет всегда выводится на HUD'е в левом верхнем углу окна.
- Доп. управление:
 - **Esc** – возврат в сцену меню. При любом возврате в сцену меню – сцена игры полностью выгружается из GameEngine.
 - ***P** – пауза.
- UI:
 - Все также остается вывод всех сущностей в игре с прошлой лабы.
 - Добавляются кнопки на переключение режимов рендера: **с текстурами** (обычный), **с коллайдерами** (отображаются только коллайдеры сущностей в виде прямоугольников или кругов), **с сеткой** (отображается сетка с ячейками 64x64 пикселя для удобной расстановки объектов на уровне. См. [ниже](#)).
 - *Добавить отдельную вкладку (tab) на UI с выводом всех ассетов, которые загружены в игру, в формате кнопок (которые пока ничего не будут делать) разбитые по типам ассетов (текстура, анимация, шрифт). В качестве кнопок должны быть изображения ассетов.
- Файл конфигурации ассетов:
 - В файле конфигурации ассетов для каждого ассета задается:

- Тип ассета (в данной ЛР – это либо **текстура**, либо **анимация**, либо **шрифт**)
 - Название ассета (строка)
 - Параметры соответствующего типа ассета (про них ниже)
- Для текстур и шрифтов параметрами является:
 - Путь до файла с текстурой или шрифта (строка)
- Для анимаций параметрами является:
 - Название используемой текстуры (строка)
 - Количество кадров анимации (short)
 - Длительность одного кадра анимации в игровых кадрах (int)
- Файл конфигурации уровня:
 - В файле конфигурации уровня задается тип объекта (сущности) и его (начальное) положение. Для этого указывается:
 - Название объекта
 - Позиция объекта в сетке (X, Y клетки – int)
- Файл конфигурации:
 - В файле конфигурации задается описание объектов (сущностей) в игре:
 - Игрока
 - Пули
 - Декораций
 - Обычного тайла
 - Кирпичного тайла
 - *Тайла с вопросом
 - Игрок описывается следующими параметрами:
 - Размеры баундинг бокса игрока (W, H в пикселях – float)
 - Скорость движения (SX – float)
 - Скорость прыжка (JY – float)
 - Максимальная скорость любого движения (SM – float)
 - Гравитация (GY – float)
 - Название базовой «текстуры» (анимации, которая будет проигрываться первая) игрока
 - Названия всех анимаций игрока
 - Пуля описывается следующими параметрами:
 - Название текстуры пули

- Скорость полета (SX – float)
- Описание Circle коллайдера пули (радиус – float)
- Декорация описывается следующими параметрами:
 - Название самой декорации
 - Название текстуры соответствующей декорации
 - По необходимости – название анимаций декорации (если они есть)
- Обычный тайл описывается следующими параметрами:
 - Название текстуры тайла
- Кирпичный тайл описывается следующими параметрами:
 - Название текстуры кирпичного тайла
 - Название анимации уничтожения кирпичного тайла (взрыва)
- *Тайл с вопросиком описывается следующими параметрами:
 - Название текстуры тайла с вопросиком
 - Название текстуры, на которую тайл с вопросом меняется после коллизии с игроком снизу
 - Название анимации, которая будет воспроизведена после коллизии с игроком снизу (анимация с монеткой)

Примеры конфигурационных файлов можно найти [ниже](#). Все примеры необходимых для выполнения задания текстур можно найти в архиве, который поставляется вместе с условием этой лабораторной. Необязательно использовать именно их, можно экспериментировать пока нет сильного ухода от описания лабы.

Как и всегда придерживаемся максимальной декомпозиции классов и методов и переиспользуем код из второй лабораторной работы.

Весь код поместить в свой публичный репозиторий на [github](#) с использованием соответствующих .gitignore и .gitattributes.

Режимы отрисовки с сеткой и коллайдерами

Сетка:

Для удобной расстановки объектов на уровне предлагается использовать сетку с ячейками 64x64 пикселя (стандартный размер одного тайла) и для ориентирования в происходящем на экране – режим отрисовки с сеткой, которая выглядит как на рисунке выше.

Данный пример показывает сетку для окна размером 1280x768. **Обратите внимание**, что нумерация ячеек идет с левого нижнего угла, опять же для удобства оформления уровня.

(0,1)	(1,1)	(2,1)	(3,1)	(4,1)	(5,1)	(6,1)	(7,1)	(8,1)	(9,1)	(10,1)	(11,1)	(12,1)	(13,1)	(14,1)	(15,1)	(16,1)	(17,1)	(18,1)	(19,1)
(0,0)	(1,0)	(2,0)	(3,0)	(4,0)	(5,0)	(6,0)	(7,0)	(8,0)	(9,0)	(10,0)	(11,0)	(12,0)	(13,0)	(14,0)	(15,0)	(16,0)	(17,0)	(18,0)	(19,0)
(0,9)	(1,9)	(2,9)	(3,9)	(4,9)	(5,9)	(6,9)	(7,9)	(8,9)	(9,9)	(10,9)	(11,9)	(12,9)	(13,9)	(14,9)	(15,9)	(16,9)	(17,9)	(18,9)	(19,9)
(0,8)	(1,8)	(2,8)	(3,8)	(4,8)	(5,8)	(6,8)	(7,8)	(8,8)	(9,8)	(10,8)	(11,8)	(12,8)	(13,8)	(14,8)	(15,8)	(16,8)	(17,8)	(18,8)	(19,8)
(0,7)	(1,7)	(2,7)	(3,7)	(4,7)	(5,7)	(6,7)	(7,7)	(8,7)	(9,7)	(10,7)	(11,7)	(12,7)	(13,7)	(14,7)	(15,7)	(16,7)	(17,7)	(18,7)	(19,7)
(0,6)	(1,6)	(2,6)	(3,6)	(4,6)	(5,6)	(6,6)	(7,6)	(8,6)	(9,6)	(10,6)	(11,6)	(12,6)	(13,6)	(14,6)	(15,6)	(16,6)	(17,6)	(18,6)	(19,6)
(0,5)	(1,5)	(2,5)	(3,5)	(4,5)	(5,5)	(6,5)	(7,5)	(8,5)	(9,5)	(10,5)	(11,5)	(12,5)	(13,5)	(14,5)	(15,5)	(16,5)	(17,5)	(18,5)	(19,5)
(0,4)	(1,4)	(2,4)	(3,4)	(4,4)	(5,4)	(6,4)	(7,4)	(8,4)	(9,4)	(10,4)	(11,4)	(12,4)	(13,4)	(14,4)	(15,4)	(16,4)	(17,4)	(18,4)	(19,4)
(0,3)	(1,3)	(2,3)	(3,3)	(4,3)	(5,3)	(6,3)	(7,3)	(8,3)	(9,3)	(10,3)	(11,3)	(12,3)	(13,3)	(14,3)	(15,3)	(16,3)	(17,3)	(18,3)	(19,3)
(0,2)	(1,2)	(2,2)	(3,2)	(4,2)	(5,2)	(6,2)	(7,2)	(8,2)	(9,2)	(10,2)	(11,2)	(12,2)	(13,2)	(14,2)	(15,2)	(16,2)	(17,2)	(18,2)	(19,2)
(0,1)	(1,1)	(2,1)	(3,1)	(4,1)	(5,1)	(6,1)	(7,1)	(8,1)	(9,1)	(10,1)	(11,1)	(12,1)	(13,1)	(14,1)	(15,1)	(16,1)	(17,1)	(18,1)	(19,1)
(0,0)	(1,0)	(2,0)	(3,0)	(4,0)	(5,0)	(6,0)	(7,0)	(8,0)	(9,0)	(10,0)	(11,0)	(12,0)	(13,0)	(14,0)	(15,0)	(16,0)	(17,0)	(18,0)	(19,0)

Пример кода, который отображает эту сетку можно найти здесь:
<https://gist.github.com/MexicanMan/2998f49a0ac635476d12213453736116>

Коллайдеры:



Для удобства понимания размеров коллайдеров в игре предлагается также добавить режим отрисовки этих коллайдеров в формате прямоугольников или кругов. Пример такого режима приведен на рисунке выше для одного объекта с Box коллайдером. Объекты без коллайдеров на экране не отображаются.

Пример кода, который отображает коллайдеры можно найти здесь: <https://gist.github.com/MexicanMan/28c54f6fa274dc3bae78049128945e28>

Примеры файлов конфигурации (json)

Ниже приведены возможные примеры того, как у вас могут выглядеть необходимые для лабораторной работы файлы конфигурации в формате (JSON).

Файл ассетов:

```
{
  "Textures": [
    {
      "name": "TexRun",
      "path": "animations/run.png"
    },
    {
      "name": "Tile",
      "path": "textures/tile.png"
    }
  ],
  "Animations": [
    {
      "name": "RunAnim",
      "textureName": "TexRun",
      "frameCount": 3,
      "frameDuration": 10
    }
  ],
  "Fonts": [
    {
      "name": "BaseFont",
      "path": "fonts/futura.ttf"
    }
  ]
}
```

Файл уровня:

```
[
  {
    "name": "Player",
    "x": 1,
    "y": 1
  },
  {
    "name": "Tile",
    "x": 0,
    "y": 0
  },
  {
    "name": "Tile",
    "x": 1,
    "y": 0
  },
  {
    "name": "Brick",
    "x": 1,
    "y": 3
  },
  {
    "name": "BigHill",
    "x": 0,
    "y": 1
  }
]
```

Файл конфигурации:

```
{
  "Player" : {
    "Bbox" : [30, 30],
    "SX": 5,
    "JY": 15,
    "SM": 20,
    "GY": 1,
    "BasePose": "IdleAnim",
    "Animations": [ "IdleAnim", "RunAnim", "JumpAnim", "ShootAnim" ]
  },
  "Decorations": [
    {
      "Name": "BigHill",
      "BaseTexture": "BigHill"
    }
  ],
  "Tile": {
    "BaseTexture": "Tile"
  }
}
```

```
    },  
    ...  
}
```